

Week 5 • Authentication, authorization, and data modeling

Technical Interaction Design • ITU • Autumn 2026

Built 24 September 2026 • commit d4807cd

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-05.pdf>

CONTENTS

Authentication and Authorization (in Parse)

Part 1: Authentication

- Parse gives you accounts, login and the current user without your writing any of it

Part 2: Authorization

- The API keys are public, so the protection has to be attached to the user
- 1. Object-Level Permissions: Access Control Lists
- 2. Class-Level Permissions
- 3. Restricting Class Creation
- Combining the three
- Field-level permissions
- Advanced: roles, for groups of users you reuse
- Reading

Relationships Between Object Classes

- The first relationship: a to-do has an owner
- The same idea has three names, depending on the context
- To-dos belong to lists
- Refactoring the schema: the owner moves to the list
- A pointer is a foreign key that nobody checks
- Many-to-many: sharing a list
- The notation matters less than being able to explain your model
- Checking the model against the screens: the CRUD matrix
- Restrictions about Parse

Authentication and Authorization (in Parse)

Source: [Lectures/Technical/Backend/Authorization-and-ACL-in-Parse.md](#) — something unclear? [Open an issue](#), or send a pull request.

Two words that sound alike and mean different things, which is why they belong in the same lecture:

- **Authentication** — proving that a user is who they say they are. *Who are you?*
- **Authorization** — deciding what that user is then allowed to do. *What are you allowed to touch?*

Here is where we are going. Ada and Armin both use your to-do app. By the end of this note, each of them logs in, and Armin cannot see Ada's to-dos, not even by skipping your app and talking to the server directly. The first half gives your app accounts (authentication); the second half stops those accounts from reading each other's data (authorization).

Part 1: Authentication

Parse gives you accounts, login and the current user without your writing any of it

On the backend, there's a separate class for user accounts

`_User` is a class that already exists in your database, with `username`, `password` and `email` already on it, and with the password already being hashed for you (stored scrambled, so that not even you can read it back).

On the front-end, `signUp` creates an account and `login` checks a password

The Javascript Parse SDK helps you manage user accounts and track the logged in user.

Signing up and logging in

```
import { useState } from 'react';
import Parse from 'parse';

export default function AuthPage({ onAuthenticated }) {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');

  async function handleSignUp(e) {
    e.preventDefault();
    setError('');
    try {
      const user = new Parse.User();
      user.set('username', username);
      user.set('password', password);
      await user.signUp();
      onAuthenticated(user);
    } catch (err) {
      setError(err.message);
    }
  }

  async function handleLogin(e) {
    e.preventDefault();
    setError('');
    try {
      const user = await Parse.User.login(username, password);
      onAuthenticated(user);
    } catch (err) {
      setError(err.message);
    }
  }

  return (
    <div>
      <h1>Welcome</h1>
      {error && <p style={{ color: 'red' }}>{error}</p>}

      <form>
        <input
          type="text"
          placeholder="Username"
        >
```

```

        value={username}
        onChange={(e) => setUsername(e.target.value)}
      />
      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <button onClick={handleSignUp}>Sign Up</button>
      <button onClick={handleLogin}>Log In</button>
    </form>
  </div>
);
}

```

What happens:

- `user.signUp()` creates a new user, and logs them in
- `Parse.User.logIn()` logs in an existing one
- Both of them manage the session for you

Information about whether the user is authenticated has to be kept as state the App level not in the authentication component

Thus

- we declare a state
- we initialize it with `Parse.User.current()`
- we pass a callback to `AuthPage` (`handleAuthenticated`) that it will call in case it has a logged in user
- then we can use [conditional rendering](#) with an early return again to either render the Auth page or the actual application

```
function App() {

  const [user, setUser] = useState(Parse.User.current());

  function handleAuthenticated(loggedInUser) {
    setUser(loggedInUser);
  }

  // conditional early return
  if (!user) {
    return <AuthPage onAuthenticated={handleAuthenticated} />;
  }

  return <ToDoList />;
}
```

Logging out

- the Parse API is offering us `Parse.User.logout()`
- we add a button
- and attach the handler

Both go in `App`, because that is where `setUser` lives:

```
function handleLogout() {
  Parse.User.logout().then(() => setUser(null));
}

// ... after the early return

return (
  <
    <ToDoList />
    <button onClick={handleLogout}>Logout</button>
  </>
);
```

The current user information is always available in `Parse.User.current()`

```
const currentUser = Parse.User.current();
if (currentUser) {
  // do stuff with the user
} else {
  // show the signup or login page
}
```

This is why `useState(Parse.User.current())` above is safe as a `useState` initializer while `fetchTodos()` was not: `current()` reads from `localStorage` and is synchronous. No network, no waiting, no third state.

How the server knows it is you

After you log in, a session token in your `localStorage` identifies you: the same `sessionToken` you can find in the backend database

Open the developer tools right after logging in, and look in **Application** → **Local Storage**. Parse has stored the current user there, and inside it a `sessionToken`: a long random string that the server handed out when your password checked out.

The `sessionToken` is the thing that compensates for the fact that HTTP is stateless

The reason the `sessionToken` exists: **HTTP is stateless**. The server forgets you the moment it has answered a request. So from now on every call your app makes carries that token, and the token is what proves who is calling. Your password was sent once, at login; the token is sent every time instead.

You can watch it travel. Open the **Network** tab, tick a checkbox, and click the request that appears. It is the same request as [last week](#), with one line more:

```
POST https://parseapi.back4app.com/parse/classes/ToDoItem/AbC123xY
{"done":true, "_method":"PUT",
 "_ApplicationId":"YOUR_APP_ID",
 "_JavaScriptKey":"YOUR_JS_KEY",
 "_SessionToken":"r:aBcD1234..."}
```

The app id and the JavaScript key say **which application** is calling, and every visitor has them. `_SessionToken` appeared when you logged in, and says **who** is calling. It is the only thing in this request that lets the server tell you apart from anybody else.

The token is not encrypted

- It is random, and long, so it can't be guessed
- Is meaningless to anyone who does not have the database.
- What protects it on its way to the server is **HTTPS**, which encrypts the whole connection.

Whoever has the token, *is you*, from the server's point of view

- Copy token out of this browser into another one, and the server cannot tell the difference.
- This is called **session hijacking**, and it is why `logout()` does not merely delete the token from your browser: it destroys the session **on the server too**.

Something unclear in this section? [Open an issue](#).

Part 2: Authorization

The API keys are public, so the protection has to be attached to the user

[Last week](#) we saw the app id and the JavaScript key written in plain text in every request your app sends. Anybody who opens your app can read them, and no setting changes that. What the server *can* tell apart is the user, through the session token from Part 1. Everything in this half hangs off that.

With your app keys, anyone can do whatever your public tables allow

- Read information that is not meant for them
- Delete your data
- Store their movie collection in your tables

When we created the database we agreed to make everything public, because we were building an MVP. Now it is time to harden it.

Parse gives you three layers of access control

Use access control in such a way that even with the keys, no harm can be done. We take the layers from the most concrete to the coarsest:

1. **Object level** — who can read and write *this row* (ACLs)
2. **Class level** — who can touch *this table* at all (class-level permissions)
3. **Schema level** — who can create *new tables* (client class creation)

Something unclear in this section? [Open an issue](#).

1. Object-Level Permissions: Access Control Lists

See the problem first

Open the app in two windows side by side: a normal one and an incognito one. Their local storage is separate, so you can be logged in as two different users at once. Log in as Ada on the left, as Armin on the right.

Ada creates a to-do. Refresh Armin's window: **Armin can see it!** Every user sees every to-do, because nothing in the database says whose it is or who may read it.

The fix: an ACL on every object

An **Access Control List** is attached to one object, and says who may do what to it. They allow you to set **fine-grained permissions** for every row in your table. Usually they are **created at the same time** as the object.

```
export const createTodo = async (text) => {
  const currentUser = Parse.User.current();

  const item = new TodoItem();
  item.set("text", text);
  item.set("done", false);

  // only the creator can read and write
  item.setACL(new Parse.ACL(currentUser));

  const result = await item.save();
  return toPlainObject(result);
};
```

Ada creates another to-do, Armin refreshes, and this one does not appear.

But the old one still does. An ACL is set on an object when it is saved; it does not reach back to objects saved before your code changed. The to-dos created without an ACL are still readable and writable by everybody, and will stay so until you give them one. For test data, the quickest fix is to delete those rows in the dashboard's database browser.

You will meet this in your own project, as old test data that behaves differently from new data.

new Parse.ACL(user) is shorthand for read / write for the given user

```
item.setACL(new Parse.ACL(currentUser));
```

is equivalent to:

```
const acl = new Parse.ACL();
acl.setReadAccess(currentUser, true);
acl.setWriteAccess(currentUser, true);
item.setACL(acl);
```

Public read, owner write: Ada's to-dos are visible to everyone, but only Ada can change them

Sometimes an object should be **read by other users** but **not written by them**. Ada is packing for a trip, and wants the people she travels with to see her to-dos, but she does not want anybody else ticking them off. `setPublicReadAccess(true)` opens reading, and leaves writing to whoever the ACL already names. This is a variant of `createTodo` above, for this example only:

```
export const createTodo = async (text) => {
  const currentUser = Parse.User.current();

  const item = new TodoItem();
  item.set("text", text);
  item.set("done", false);

  const acl = new Parse.ACL(currentUser); // Ada can read and write
  acl.setPublicReadAccess(true);           // everybody can read
  item.setACL(acl);

  const result = await item.save();
  return toPlainObject(result);
};
```

Armin refreshes, and Ada's to-do appears in his list. He ticks its checkbox. What happens?

- **The database does not change. The screen says it did.** Open the **Network** tab and look at the response to Armin's request: `{"updatedAt": "..."}`, the same answer a successful save gets. `save()` resolves, the state updates, and the checkbox shows a tick. Refresh, and the tick is gone: the ACL did its job, and the row was never written. The server just did not tell anybody. (Back4App answers every update this way when it matches no row, even an update to an id that does not exist.)
- **Delete is more honest.** Try deleting Ada's to-do from Armin's window: the server refuses with error 101, **"Object not found"**. Not "permission denied": the server does not even admit that a row Armin may not touch exists.
- **The server is protected, but the interface is not honest.** Armin was offered a checkbox that could never work, and because the server reports success, the app cannot even find out afterwards. It has to know *before*.

An ACL can be changed every time the object is saved, so Ada can open her to-dos up, or close them again, later.

Ask the ACL before drawing the checkbox

Every object comes back from the server with its ACL, so the client can find out, before drawing anything, whether the current user may change it. The place for that is `toPlainObject`, from [last week's services file](#): it already decides what a to-do looks like to React, so it gets one more field.

```
function canWrite(parseObject) {
  const acl = parseObject.getACL();
  if (!acl) return true;           // no ACL at all: everybody may write

  const user = Parse.User.current();
  return acl.getPublicWriteAccess() || acl.getWriteAccess(user);
}

function toPlainObject(parseObject) {
  return {
    id: parseObject.id,
    text: parseObject.get("text"),
    done: parseObject.get("done"),
    canWrite: canWrite(parseObject),
  };
}
```

Both checks are needed. `getWriteAccess(user)` only looks for the user's *own* entry in the ACL, so on an object that is writable by everyone it still says `false`.

Then the component uses it. A disabled checkbox still shows whether Ada has done the to-do; Armin just cannot change it. A button that could never work is better not drawn at all:

```
<li>
  <input
    type="checkbox"
    checked={elem.done}
    disabled={!elem.canWrite}
    onChange={() => onChange(elem.id)}
  />
  {elem.text}
  {elem.canWrite && (
    <button type="button" onClick={() => onDelete(elem.id)}>Delete</button>
  )}
</li>
```

This is for the interface only. The client is running on Armin's computer, and nothing stops him from sending the request anyway, with `curl` or with the dev tools. What refuses it is still the ACL on the

server. `canWrite` does not protect anything; it only stops the app from offering something it cannot do. (It also ignores roles. If your app puts roles in ACLs, `canWrite` has to check those too.)

Sharing with another user

```
const acl = new Parse.ACL(currentUser); // owner has full access
acl.setReadAccess(otherUser, true);      // other user can read
acl.setWriteAccess(otherUser, true);     // other user can write
```

`otherUser` is a `Parse.User`, and if all you have is their id, `Parse.User.createWithoutData(id)` makes one. How your app learns that id in the first place (an invitation, a share link, a user who accepts a shared list) is a design question for your own app. It gets harder, not easier, once you lock down the `_User` class below.

Sharing a whole list, rather than one object, also needs the database to *know* who it is shared with, which an ACL cannot tell you: [Many-to-many: sharing a list](#).

Something unclear in this section? Open an issue.

2. Class-Level Permissions

Our login page protects nothing: anyone can skip the app and talk to the server directly

Our app now shows nothing but the login page to a visitor who is not logged in. That conditional render feels like a lock, but it only locks our own interface. The server is one HTTP request away, and the keys that request needs are the ones we ship to every visitor. So skip the app entirely, and create a to-do from the terminal, without ever logging in:

```
curl -X POST "$PARSE_SERVER_URL/classes/ToDoItem" -H "X-Parse-Application-Id: $APP_ID" -H "X-
Parse-Javascript-Key: $JS_KEY" -H "Content-Type: application/json" -d '{"text":"I
never logged in","done":false}'
```

`curl` sends an HTTP request from the terminal. Replace `$PARSE_SERVER_URL`, `$APP_ID` and `$JS_KEY` with the three values from your `.env.local`.

It works. The server answers with the `objectId` of a new row, and because nobody attached an ACL to it, the row is public: it shows up in the list of every user of your app.

ACLs cannot stop this, because an ACL belongs to a row that already exists. Stopping the *creation* is a job for the next layer: class-level permissions sit in front of the ACLs and decide whether a user may touch the class *at all*.

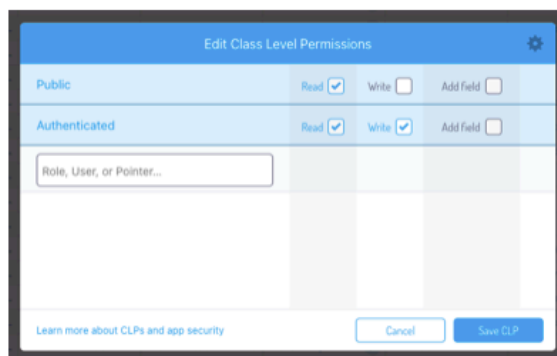
For every class you choose who has access and with which privileges.

Who has access can be specified with multiple levels of granularity

- Public (anyone, even unauthenticated)
- Authenticated users (the **Authenticated** row in the dashboard)
- Specific users
- Roles

Privileges for the entire class

The dashboard's *Edit Class Level Permissions* dialog opens in a simple view, with three columns:



- **Read** — retrieve objects of this class
- **Write** — create, change and delete objects of this class
- **Add field** — add new fields to the schema

Read and Write are each a group of finer privileges. The gear icon in the top right corner switches to the detailed view, with one checkbox for each:

- Read = **Get** (one object, by its id) + **Find** (queries) + **Count**
- Write = **Create** + **Update** + **Delete**

Try it: only logged-in users may create to-dos

Somebody without an account has no business creating to-dos. In the dashboard:

1. Open the database browser and select the `TodoItem` class.
2. Open its **Security** settings. The *Edit Class Level Permissions* dialog above appears.
3. In the **Public** row, untick **Write**. In the **Authenticated** row, keep **Read** and **Write** ticked.
4. Click **Save CLP**.

Then run the same `curl` again: this time the server refuses. The app still works for Ada and Armin, because they are logged in.

Unticking **Write** for the public takes away Create, Update and Delete together. To take away only Create, use the detailed view.

One toggle, and one layer of the lecture becomes visible. Class level says who may knock; the ACL says which rows open.

The exception: `_User`

Every other class tightens. One class cannot: `_User` must keep **Create public**, because signing up is creating a user, and the person signing up is by definition not logged in yet.

What must not stay public on `_User` is everything else. By default a user can only modify their own user object, but user objects can be read by anyone — so with the keys from your bundle, a stranger could list every account in your app. In the `_User` class-level permissions, switch to the detailed view with the gear icon, and untick **Find** in the **Public** row. Create stays open.

3. Restricting Class Creation

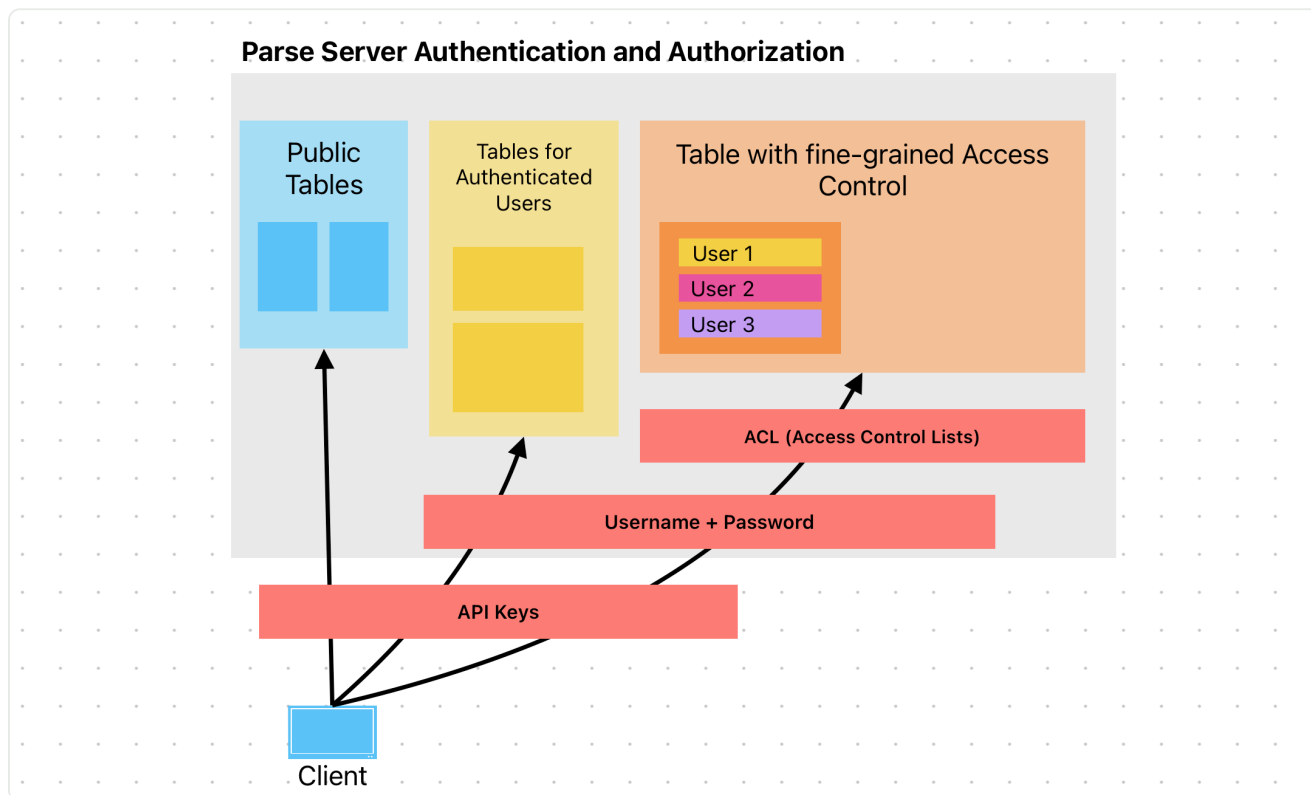
Surely, not all users should be allowed to create classes either!

Under `App Settings > Server Settings > Client Class Creation` you can specify if you expect users to be allowed to create new classes in your database. Very handy in week one, when saving to a class that does not exist yet simply creates it. Very dangerous in production, where anyone with your keys can do the same. Turn it off.

Something unclear in this section? [Open an issue](#).

Combining the three

The methods above should be combined together to strengthen the DB access for your application.



Most classes should not be public. When some data really should be readable by everyone, like Ada's to-dos above, make that a decision per object, with an ACL, rather than a default for the whole class. And data that never changes, like a list of countries, may not need a class at all: it can live in the app's code.

Something unclear in this section? [Open an issue.](#)

Field-level permissions

An ACL covers a whole object, so it cannot make one field public and another private. When you need that — a to-do whose text is shared but whose time-tracking is not — the data has to be split across two classes. That is week 12's subject: [Field-Level Permissions, and the Two-Table Workaround](#).

Something unclear in this section? [Open an issue.](#)

Advanced: roles, for groups of users you reuse

If your design has **named groups of users that are reused** across many objects — a family, a team, the moderators — look up **roles**: a role is an object that holds users, and you can put a role into an ACL instead of listing the users one by one. For one-off sharing with a person or two, putting them in the ACL directly is simpler.

Create the role once, then use it in as many ACLs as you like

Ada wants her family to see and change her shopping to-dos. Instead of naming every family member in every ACL, she creates a role for the family, and puts the role in the ACLs:

```
// 1. The role. Its own ACL says who may change who is in it: only Ada.
//    Role names are unique across the whole app, hence the id in the name.
const family = new Parse.Role("Family-" + currentUser.id, new Parse.ACL(currentUser));
family.getUsers().add(armin);    // armin is a Parse.User
await family.save();

// 2. A to-do that the whole family may read and change
const acl = new Parse.ACL(currentUser);
acl.setRoleReadAccess(family, true);
acl.setRoleWriteAccess(family, true);
item.setACL(acl);
await item.save();
```

The payoff comes later. When Ada adds her sister to the role, her sister can see every to-do with `family` in its ACL at once, and not a single ACL has to change :)

Two types of roles

Application-level roles (Moderators, Admins, Premium Users)

- Created manually in Parse Dashboard or via Cloud Code
- Managed by the app administrators, not end users
- Public read of the role is normal - users should see who the moderators are

User-created roles (e.g. the family above, a team, a project group)

- Created by regular users from the app, as above
- Each user manages their own
- Private - no reason for others to see them

Something unclear in this section? Open an issue.

Reading

From the ParsePlatform.org Guide:

- [Class-level Permissions](#)
- [Object-level Access Control](#)

Exam Questions

1. Why can't Parse API keys be kept secret in a web application?
2. What is the difference between authentication and authorization?
3. HTTP is stateless. How does the server know, on your tenth request, that it is still you? What happens if someone copies that thing into their own browser?
4. What are the three layers of access control in Parse, and what does each one protect?
5. What is an ACL and at what level does it operate?
6. Explain what this code does:

```
const privateNote = new Note();
privateNote.set("content", "My secret");
privateNote.setACL(new Parse.ACL(Parse.User.current()));
await privateNote.save();
```

7. How would you make an object publicly readable but only writable by the owner?

8. User B, logged in, can see a to-do that user A created. Give every reason that could be the case.

9. Why must the `_User` class keep `Create` public, when every other class should not?

10. When would you reach for a role instead of listing users in an ACL?

Something unclear in this section? [Open an issue](#).

Relationships Between Object Classes

Source: [Lectures/Technical/Backend/Relationships-Between-Object-Classes.md](#) — something unclear? [Open an issue](#), or send a pull request.

Until today there was one class, and no modeling was required: a to-do has a text and a done flag, and that is the whole design. This note is about what you have to decide the moment there is a second kind of thing in the application: users who own to-dos, or lists that to-dos belong to. How two classes refer to each other is the decision that is hardest to undo once there is data in the database. It assumes you have already written [the four operations against one class](#), and it follows straight on from [Authentication and Authorization](#).

You need to think ahead about the database model as soon as there is more than one kind of thing in your application. The main questions are:

1. What are the types of objects in my domain model?
2. What are the relationships between them?

The first relationship: a to-do has an owner

After the authorization lecture every to-do has an ACL that says Ada may read it. Does that mean the database knows the to-do is *Ada's*?

Not in any way you can use. The ACL is a permission, not a relationship: you cannot write a query that says “the to-dos whose ACL mentions Ada”. The ACL only filters, silently, what comes back. If the app wants to *ask* for Ada's to-dos, the to-do needs a field that says whose it is:

```
item.set("owner", Parse.User.current());
```

and then the query can say it out loud:

```
const query = new Parse.Query(TodoItem);
query.equalTo("owner", Parse.User.current());
const results = await query.find();
```

Run it in the two-browser setup from last lecture and notice the difference from before: until now Armin saw his own to-dos *plus* the old ones nobody had put an ACL on. Now he sees only the ones that are his, because now he is asking for exactly those.

Remember which of the two is the security, though. The query is something *our* client chooses to send, and nothing obliges anyone else to send it. They do not even need your client: as the `curl` request in [Authentication and Authorization](#) showed, the keys you shipped are enough to send any request, any query included. What stops them is the ACL, on the server.

That does not make the query a mere convenience. It is also the app's logic: it says what this screen is *for*. "My to-dos" should show Ada's to-dos, and it should keep showing only those even on the day an ACL is set wrong, or once lists start being shared with her. The ACL decides what Ada *may* see; the query decides what she *asked* to see.

Something unclear in this section? [Open an issue](#).

The same idea has three names, depending on the context

In a relational database	In Parse	In OO lingo
table	class	class
row	object	object
column	field	attribute
foreign key	pointer	a reference
join table	a class with two pointers	—

So a **pointer** is Parse's **foreign key**. It holds which object in which class, and nothing else.

A to-do's `owner` field holds a pointer to a `_User`, Parse's built-in class for user accounts.

Something unclear in this section? [Open an issue](#).

To-dos belong to lists

A single flat pile of to-dos stops being useful somewhere around thirty items. What people actually want is *Personal*, *Apartment*, *Bachelor project*. Several lists, each with a name.

Note what we do *not* do: we do not add a `list` string field to `TodoItem`.

A string would be enough to group them on screen today, and it would be inefficient the moment anyone wants to rename a list.

This is called `normalization` in databases.

If a concept is expressed in a single place, it's easy to change. In our case, renaming a list: we rename it in a single place.

Benefit of a class/table over an attribute is that the class/table can be enriched with more properties later

Our lists could get new properties: priority, etc. Moreover, you may want to share a list with other users. If a list is a first class entity in the DB that becomes easily possible ([Many-to-many: sharing a list](#), below).

Create the first list by hand

Before writing any code for lists, open the dashboard, create a `List` class with a `name` column (String) and an `owner` column (Pointer to `_User`), and add one row by hand: *Apartment*, with `owner` set to your own user's `objectId`. Without the owner, the list will not show up once the app loads "my lists" further down.

The dashboard and your app talk to the same server, so a list created there is exactly as real as one created from code. That is a genuinely useful habit for your project: **not every class needs a create screen in version one**. Seed the things that rarely change by hand, and build screens for what your users actually create. A create screen is worth building when creating that thing is part of the use case you are designing for.

One-to-many relationships are done with pointers

One list has many to-dos; each to-do belongs to exactly one list. That is a **one-to-many** relationship, and the pointer goes on the *many* side: each to-do stores one pointer to its list. The other way round, the list would need an array of to-dos, and the note on arrays at the end says why not. The rule of thumb: **pointers for one-to-many, a join table for many-to-many**.

Set a pointer by handing `set()` the whole object, not its id:

```
const List = Parse.Object.extend("List");
const TodoItem = Parse.Object.extend("TodoItem");

// the list you created in the dashboard, by its objectId
const apartmentList = await new Parse.Query(List).get("<objectId from the dashboard>");

const item = new TodoItem();
item.set("text", "Call the landlord");
item.set("done", false);
item.set("list", apartmentList); // ← the object itself, not its id
await item.save();
```

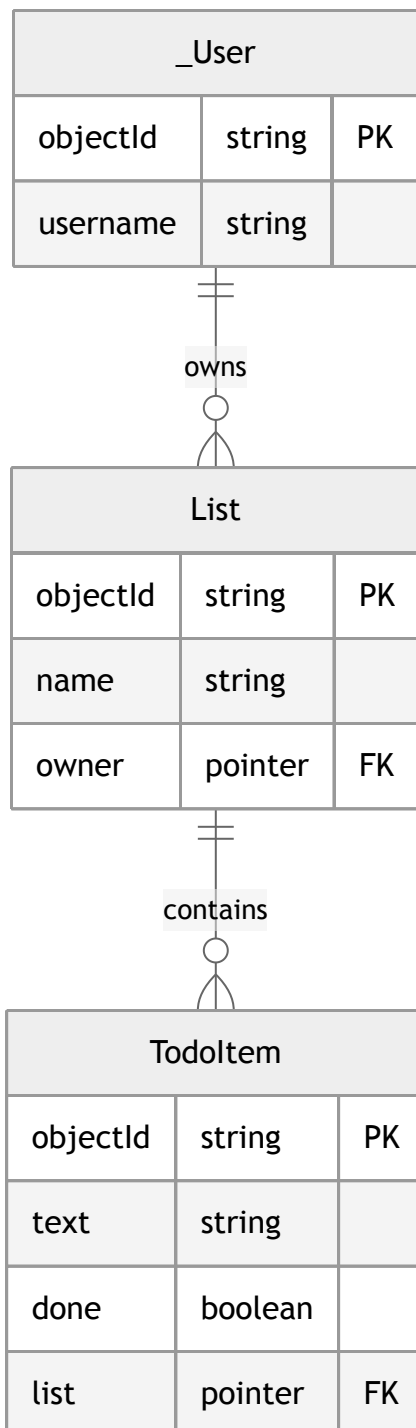
Now we can query all the to-dos in a list:

```
const query = new Parse.Query(TodoItem);
query.equalTo("list", apartmentList);
const results = await query.find();
```

Something unclear in this section? Open an issue.

Refactoring the schema: the owner moves to the list

Now look at the model again. The list belongs to Ada; every to-do in it belongs to Ada. The `owner` on each to-do says the same thing a hundred times. So the owner moves to the list, and the to-do reaches its owner through it:



Read `||--o{` as "one to many": one user owns many lists, one list contains many to-dos. The notation is explained at the end of this note.

Then drop the `owner` column from `TodoItem` in the dashboard. It is gone, instantly. Nothing complains, because nothing is enforcing anything: the existing rows simply lose the field. Schema

changes are this easy in Parse *precisely because* the database checks nothing. Freedom and footgun, same coin.

Two things stop working when the column goes. The “my to-dos” query from the start of this note (`equalTo("owner", ...)` on `TodoItem`) now returns nothing: to find Ada’s to-dos, the app now finds her lists, and then the to-dos in each (below). And to-dos created before lists existed have no `list` pointer, so no list shows them. Give them one in the dashboard, or delete them.

The ACLs stay where they are, on every list and every to-do. **Parse does not pass an ACL down from a list to its to-dos.** If the to-dos should follow their list, your code sets the same ACL on both. That matters the day you share a list: it is the list *and* its to-dos that have to change. So a new to-do copies the ACL of its list:

```
export const createTodo = async (text, list) => {
  const item = new TodoItem();
  item.set("text", text);
  item.set("done", false);
  item.set("list", list);
  item.setACL(list.getACL()); // the to-do follows its list
  return toPlainObject(await item.save());
};
```

Creating lists from the app

The cheapest honest version: one text input and a *New list* button at the bottom of the page.

```
export const createList = async (name) => {
  const currentUser = Parse.User.current();

  const list = new List();
  list.set("name", name);
  list.set("owner", currentUser);
  list.setACL(new Parse.ACL(currentUser));

  return await list.save();
};
```

We have no router yet, so every list renders on the same page, one block per list. Each block gets its **own** new-to-do input. Which list a new to-do belongs to is decided by where the input sits: the list object is already in scope when you render its block, so there is no dropdown and no “selected list” state to keep in sync.

Loading the page, the simple way

```
const listQuery = new Parse.Query(List);
listQuery.equalTo("owner", Parse.User.current());
const lists = await listQuery.find();

// and then, inside each list's block:
const todoQuery = new Parse.Query(TodoItem);
todoQuery.equalTo("list", list);
const todos = await todoQuery.find();
```

It works. The page is also getting crowded. That is the problem routing solves, next week.

Something unclear in this section? [Open an issue.](#)

A pointer is a foreign key that nobody checks

In your database course, a foreign key came with a promise: the database will not let it point at a row that does not exist. Parse makes no such promise. Delete a list, and its to-dos are still there, each with a `list` pointer to nothing.

If you want the guarantee, you write it yourself, on the server, in a trigger that runs whenever a list is deleted. See `beforeDelete` in [Running Code Server-Side](#). The guarantee moves out of the database and into your code.

Something unclear in this section? [Open an issue.](#)

Many-to-many: sharing a list

Ada wants to share *Apartment* with Armin, who is moving in with her. A list can be shared with many users, and a user can have many lists shared with them. That is a **many-to-many** relationship. Every project in this course has one, because sharing at the list level is part of the required core.

Sharing with the ACL alone

The simplest version needs no new class. Sharing is [Sharing with another user](#), applied to the list and its to-dos, because [Parse does not pass an ACL down](#):

```
export const shareList = async (list, friend) => {
  const acl = list.getACL();
  acl.setReadAccess(friend, true); // Armin may read, not change
  list.setACL(acl);

  const todos = await new Parse.Query(TodoItem).equalTo("list", list).find();
  todos.forEach((todo) => todo.setACL(acl));

  await Parse.Object.saveAll([list, ...todos]);
};
```

Finding the friend: by their exact username

`shareList` needs Armin as a `Parse.User`. The smallest way to get him: one text input where Ada types his username, exactly.

```
const friend = await new Parse.Query(Parse.User)
  .equalTo("username", name)
  .first(); // undefined if nobody has that username

if (!friend) {
  setError("No user with that name");
  return;
}

await shareList(list, friend);
```

No search, no autocomplete, no list of everybody who has an account. That is less code, and it is also better privacy: Ada can only find someone whose username she already knows. It needs **Find** on `_User` for the **Authenticated** row. The [Auth note](#) takes Find away from the public, not from logged-in users.

Showing it: every list I can see

The page drops the `owner` filter. It asks for every list, and the ACL decides which ones come back: Ada's own, and the ones shared with her. The owner, brought along with `include()`, tells them apart:

```
const query = new Parse.Query(List); // no owner filter: the ACL decides
query.include("owner"); // bring the owner object along, not only its id
const lists = await query.find();

// for each list:
const mine = list.get("owner").id === Parse.User.current().id;
// mine ? "Mine" : `Shared by ${list.get("owner").get("username")}`
```

This is a legitimate design, and for many apps it is enough. Know what it costs, though. The query no longer says what the screen is for, so **every ACL mistake is now on screen**. Take a list created in the dashboard without an ACL. It is publicly readable, so it shows up for every user of your app, as "shared by" whoever owns it.

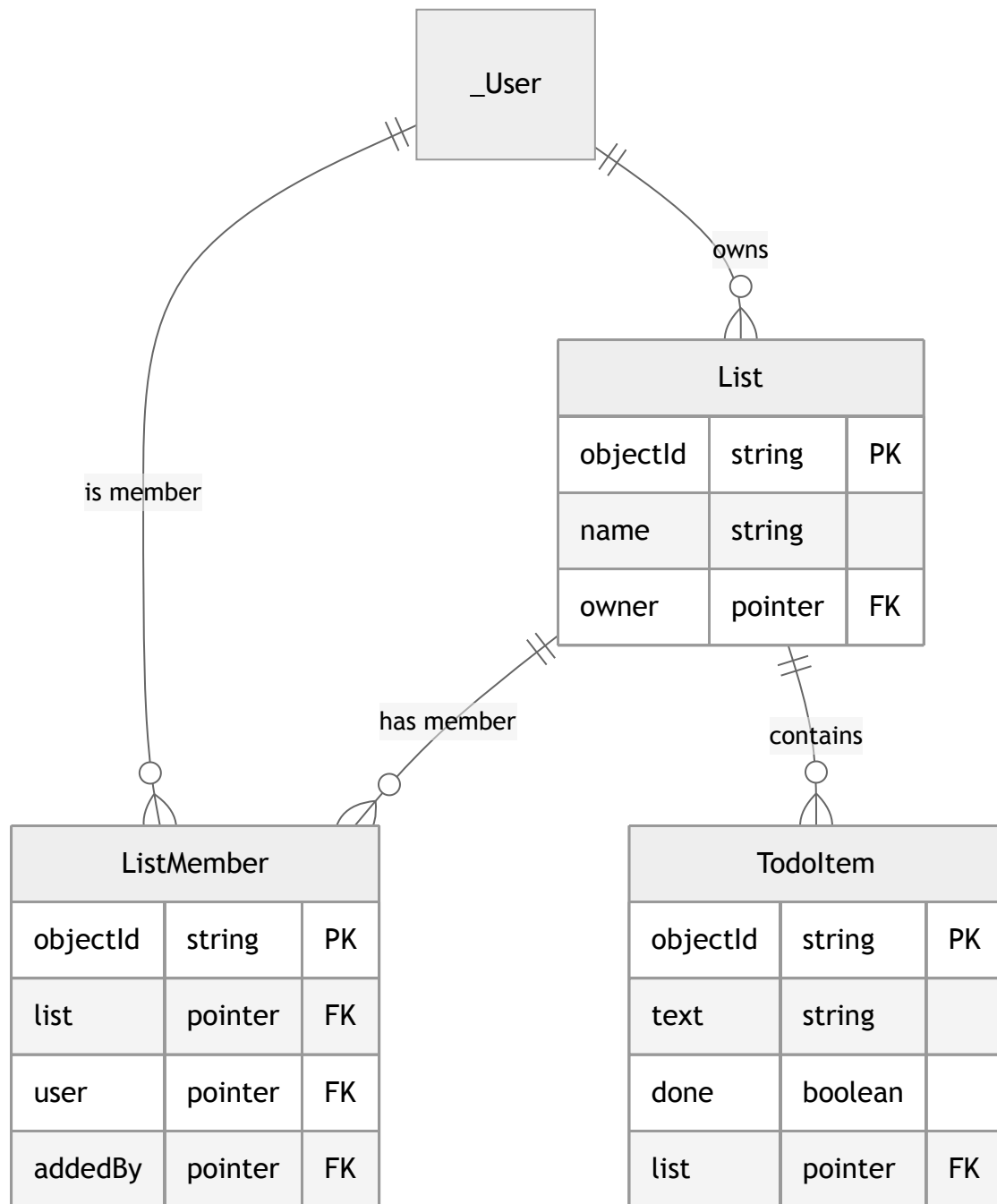
When the relationship needs to become a class: Armin wants to leave

Armin is done with the apartment, and wants *Apartment* off his screen. He cannot do it. The fact "Armin may read this list" lives inside the list's ACL. The list is Ada's, and Armin only has read access to it. Only Ada can take him off.

That is the sign. **When someone other than the owner needs to change a relationship, the relationship needs a row of its own.** That someone can then be given the right to change the row. The same goes for anything you need to *store* about the relationship: an invitation Armin has to accept first, who added him, whether he may edit.

A join table: ListMember

Create a class whose job is to represent one membership: one row says *this user is on this list*.



Two pointers, one per side. That is all a join table is: two one-to-many relationships, meeting in the middle. Sharing now also writes a membership. It has an ACL of its own, which lets **both** of them change it:

```
const ListMember = Parse.Object.extend("ListMember");

const member = new ListMember();
member.set("list", list);
member.set("user", friend);
member.set("addedBy", Parse.User.current());
const memberAcl = new Parse.ACL(Parse.User.current()); // Ada
memberAcl.setReadAccess(friend, true);                 // and Armin
memberAcl.setWriteAccess(friend, true);
member.setACL(memberAcl);
await member.save();
```

Armin's shared lists and Ada's members are now ordinary queries. And leaving is one line, which Armin is allowed to run:

```
// Armin: the lists shared with me
const shared = new Parse.Query(ListMember);
shared.equalTo("user", Parse.User.current());
shared.include("list");
const memberships = await shared.find();

// Ada: who is on this list?
const members = new Parse.Query(ListMember);
members.equalTo("list", apartmentList);
members.include("user");
const people = await members.find();

// Armin leaves
await membership.destroy();
```

The same shape fits any many-to-many: labels on to-dos would be a `TodoLabel` class, with a `todo` and a `label` pointer.

Leaving is recorded in the data; access is still decided by the ACL

Armin has destroyed his membership, and the list is off his screen. But the ACLs on the list and its to-dos still name him, and the server still lets him read them. `ListMember` is data; it opens and closes nothing.

So the same fact now lives in two places: `ListMember` says who is on the list, the ACLs say who may read it. That is the redundancy argument from [the refactoring above](#). This time it cannot be removed, because one copy is the data and the other is the security. Sharing has the same problem the other way round. `saveAll` sends the membership and the ACLs in one request, but not as a transaction. If it fails halfway, they disagree.

Keeping the two in step needs code with authority over Ada's objects, even when Armin is the one acting. That is the server. An `afterDelete` on `ListMember` that takes Armin out of the ACLs is exactly what [Running Code Server-Side](#) is for. Until then, the client does its best, and you know where the gap is.

Is `owner` still needed?

Once `ListMember` exists, Ada could be a member of her own list too, and the `owner` pointer would go. Or `owner` stays, because *the one who may delete the list and share it* is a different thing from *a member who reads it*. Both are defensible; which one is right depends on what your users do. It is exactly the kind of decision the data-model page of your report asks you to explain.

Where sharing grows from here, and why to keep it small

Everything past this point gets expensive quickly: a list of members, removing one, co-owners, members who may invite others, handing a list over, groups of people you share with again and again.

Each of them is mostly **interface work**: screens to find, add, show and remove people. And each of them has to keep the memberships and every ACL in step. The tools exist: [roles](#) put one group in an ACL instead of every person, and server code keeps the copies agreeing. But none of it is in the required core.

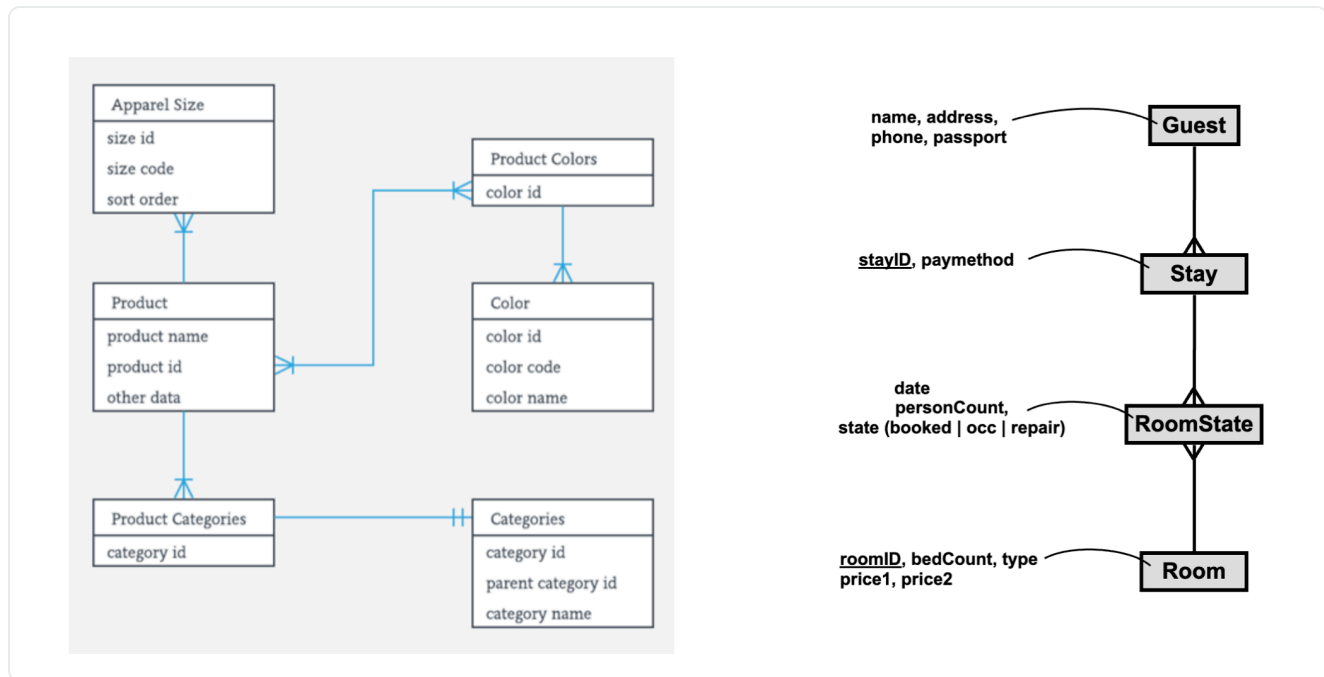
Sharing is the most expensive feature in your app, and almost all of the cost is the interface. Build the smallest version your user study supports: share by username, and show who shared it. Treat everything beyond that as an optional feature you pick and defend.

Something unclear in this section? [Open an issue](#).

The notation matters less than being able to explain your model

Use whichever notation you prefer. Two that I like are:

1. On the left hand side is the most popular way of showing attributes
 - crow's feet show cardinality
 - attributes are listed in the box
2. On the right hand side is a compressed approach proposed by Søren Lauesen, ex-professor at ITU



No matter which notation you use, the most important aspect is being able to communicate the way all the relevant data for your application domain is saved in the database.

Something unclear in this section? Open an issue.

Checking the model against the screens: the CRUD matrix

Write your classes down the side and your screens across the top. In each cell, note whether that screen lets the user **Create**, **Read**, **Update** or **Delete** that class.

	Main page
TodoItem	C R U D
List	C R

Every class your users own should be fully covered *somewhere*. Here the gap is obvious: a list can be created and shown, but never renamed or deleted.

And the moment you add *delete list*, the previous section comes back: what happens to its to-dos? Decide: delete them too, or refuse to delete a non-empty list. Then enforce it on the server.

Do this for your project model, against your wireframes. It is the cheapest way to find the screen you forgot.

Something unclear in this section? [Open an issue.](#)

Restrictions about Parse

Do not model relationships with `Parse.Relation`

You will meet `Parse.Relation` in the documentation, which is Parse's built-in way of doing many-to-many. It is less typing, and it cannot carry any information about the relationship, or have an ACL of its own that lets Armin leave. So we are not going to use it. Knowing that it exists is enough.

Do not model relationships with arrays

Parse lets you store an array of objects in a field, so a `members` array on the list is tempting. Resist it. An array has no room for information about the relationship. It lives inside the list, so only the list's owner can change it. It has to be rewritten in full to add one element. And it gets slow and awkward as soon as it is not tiny. Pointers for one-to-many, a join table for many-to-many. Those two cover everything you need this semester.

Something unclear in this section? [Open an issue.](#)

Exam Questions

1. What are the two questions you have to answer about your domain before you create any classes?

2. A to-do already has an ACL that only lets its creator read it. Why does it still need an `owner` pointer?

3. At the start of this note, this query returned only the current user's to-dos. Explain why it was nevertheless not a security measure, and what is. Why do we still write it?

```
const query = new Parse.Query(TodoItem);
query.equalTo("owner", Parse.User.current());
const results = await query.find();
```

4. Your app has lists and to-dos. Which class gets the pointer, and why not the other one?
5. Why model a list as its own class rather than as a string field on each to-do?
6. You delete a list. What happens to its to-dos in Parse, and how would it differ in the relational database from your database course? How would you get the guarantee back?
7. Why is a join table preferred over an array field or a `Parse.Relation` for list membership?
8. Ada shared a list with Armin through its ACL. Armin wants to leave it. Why can he not, what does a `ListMember` class change, and what does it still not change?
9. Once `ListMember` exists, do you still need an `owner` pointer on `List` ? Argue for one answer.

References

The documentation on ParsePlatform.org

- [Relationships](#) - this is very good and must be read attentively – it will really help with modeling